

Qué falta para continuar la app móvil

14 de julio de 2026 · Reemplaza la "Solicitud de datos v2" · **Leyenda:** ● bloqueante · ● esta semana · ● puede esperar

1. Dónde estamos

El esqueleto de la app ya existe y funciona: Expo + TypeScript en `apps/mobile`, 3 pestañas (Inicio · Mis rentas · Perfil), búsqueda por texto, detalle de artículo con calendario de disponibilidad y estados de reserva con sus colores. Todo el código ya usa **la forma del contrato del PR #2** (snake_case, USD en centavos, enums en inglés, rangos de fechas), así que no habrá que rehacerlo cuando el contrato se apruebe. Los datos hoy son simulados detrás de una interfaz (`FuenteDatos`): al llegar el contrato congelado se escribe el cliente real y las pantallas no se tocan.

La rama `feature/mobile-scaffold` está lista en local, pendiente de mostrarse al equipo antes de subirla.

2. Lo que necesito de TRUCY (Backend · dueña del contrato)

2.1 ● Cerrar y congelar el contrato (PR #2)

Es LA dependencia: sin contrato congelado no se puede escribir el cliente real ni el flujo de reserva. Antes de aprobar, propongo resolver estas observaciones que salieron de revisar el borrador:

#	OBSERVACIÓN	POR QUÉ IMPORTA	PROPUESTA CONCRETA
1	El schema <code>Error</code> es solo un texto (<code>{"error": "mensaje"}</code>), sin códigos estables. El <code>CANNOT_RENT_OWN_ITEM</code> que ya habíamos acordado no aparece.	Sin códigos, la app tendría que adivinar el error comparando textos: frágil y se rompe con cualquier cambio de redacción.	Agregar campo <code>code</code> : <code>{"error": {"code": "DATES_UNAVAILABLE", "message": "..."} }</code> . Mínimo: <code>CANNOT_RENT_OWN_ITEM</code> , <code>DATES_UNAVAILABLE</code> , <code>UNAUTHORIZED</code> , <code>TOKEN_EXPIRED</code> , <code>NOT_FOUND</code> .
2	<code>LoginResponse</code> trae solo <code>access_token</code> , sin refresh ni duración declarada.	Cuando el token expire, la persona queda deslogueada a mitad de uso (p. ej. durante un check-in).	O refresh token, o declarar duración larga para el curso (p. ej. 24 h) y documentarlo en el spec.
3	<code>POST /items/{id}/reservations</code> sin idempotencia.	Doble tap o reintento por mala señal = dos reservas idénticas.	Header <code>Idempotency-Key</code> (UUID generado por la app), o que el backend rechace duplicado exacto (mismo <code>renter+item+fechas</code>) con 409.
4	La descripción de <code>unavailable_dates</code> menciona estados en español ("solicitada, aprobada...") pero el enum es en inglés.	Detalle de consistencia; confunde a quien lea el spec.	Corregir la descripción a <code>requested</code> , <code>approved</code> , <code>delivered</code> , <code>returned</code> .

2.2 ● Tres respuestas cortas que el contrato aún no da

1. ¿ `category` acepta una sola o varias (`?category=tools,camping`)? — define si mis chips son de selección única o múltiple.
2. ¿ `min_price` / `max_price` son inclusivos y se puede mandar solo uno? — lo necesito para los chips `< $10` y `> $30` .
3. Para el polling: ¿cada cuánto es razonable consultar `GET /reservations/{id}` sin castigar al backend? (propongo cada 15 s con la pantalla abierta).

2.3 ● API corriendo

¿Cuándo habrá un API contra el cual apuntar la app, aunque sea local (`docker compose up`) o en un dev compartido? La app se configura con `EXPO_PUBLIC_API_URL` , solo necesito el valor.

3. Lo que necesito de WA (Plataforma / QA)

PRIORIDAD	QUÉ	DETALLE EXACTO DE LO QUE PIDO
●	Activar rulesets en GitHub	En Settings → Rules, sobre <code>develop</code> y <code>main</code> : requerir PR (bloquear push directo), 1 aprobación mínima, check <code>ci-gate</code> en verde y review de Code Owners. Hoy las reglas del equipo son de palabra: el PR #1 se pudo haber mergeado a <code>main</code> sin revisión. Con proceso = 50% de la nota, es lo más urgente del proyecto.
●	Corregir CODEOWNERS	Reemplazar los alias por los usuarios reales: <code>@josepicado95</code> (Trucy), <code>@josmedina</code> (Silverk), <code>@psced10-creator</code> (Zero), <code>@Wa39</code> (Wa). Sin esto, el ruleset de Code Owners no tiene a quién exigir.
●	Servicio de imágenes (URLs prefirmadas)	Para la foto de check-in/check-out (y la del reporte, que el contrato exige): ¿qué endpoint pido?, ¿qué devuelve (<code>upload_url</code> + <code>public_url</code>)?, ¿subo con PUT o multipart y con qué headers?, ¿cuánto dura la URL?, ¿límites de tamaño/formato?, ¿requiere token? Plan B si no llega esta semana: acordar un <code>photo_url</code> placeholder para no bloquear el flujo.
●	Depósito mock	Confirmar si el hold/release/freeze lo dispara solo el backend (mi lectura del contrato: sí, en approve/close/report) o si la app debe llamar algo. Si hay endpoint propio de pagos, necesito su spec.
●	Seeds en <code>infra/</code>	Un dueño y un arrendatario con credenciales conocidas; 8-10 items con foto, categorías variadas y precios distintos (para probar los chips de rango); reservas ya creadas en varios estados (para probar Mis rentas sin recorrer el flujo entero cada vez).
●	CI del móvil	¿Qué correrá <code>ci-gate</code> sobre <code>apps/mobile</code> : lint, tsc, test, build? ¿Config de ESLint/Prettier que deba respetar? Ya dejé <code>npm tsc --noEmit</code> y <code>npm expo export</code> pasando — puedo exponerlos como scripts npm con los nombres que quieras.
●	Registrar el stack	Actualizar el "Stack TBD" del CLAUDE.md: móvil = React Native + Expo (SDK 57, expo-router, TypeScript), ya materializado en el scaffold.
●	Revisar/mergear PR #1	Es solo documentación (preguntas y spec de búsqueda); tenerlo en develop evita que las decisiones se pierdan.

4. Lo que necesito de SILVERK (Web del dueño)

- **Artículos de prueba publicados desde su dashboard** en cuanto su app exista (mientras tanto me sirven los seeds de Wa). La demo final exige el flujo: **dueño publica en web → arrendatario renta en móvil**.
- **Acordar la visibilidad cruzada del reporte de problema**: cuando el arrendatario reporta desde el móvil, ¿qué ve el dueño en la web (y viceversa, si el dueño reporta en la devolución)? Propuesta mínima: evento en el historial + badge de depósito congelado en ambos lados.
- **Sincronizar etiquetas en español** de estados y categorías para que web y móvil digan lo mismo (mi mapeo está en `apps/mobile/src/data/estados.ts` — se lo puedo pasar).

5. Decisiones que me tocan a mí (Zero)

1.  Revisar y **aprobar el PR #2** (contrato) en GitHub, idealmente después de que Trucy responda las observaciones de la sección 2.1.
2.  Enseñar el scaffold al equipo y, con su visto bueno, **subir la rama y abrir el PR** del scaffold hacia `develop`.
3.  Aprobar el PR #1 (mis propios docs) cuando Wa lo revise.

6. Qué construyo en cuanto llegue cada cosa

EN CUANTO LLEGUE...	CONSTRUYO...
Contrato congelado (Trucy)	Cliente real de la API implementando la interfaz <code>FuenteDatos</code> + pantallas de login/registro con el token en <code>expo-secure-store</code> .
API corriendo (Trucy/Wa)	Conexión real: listado, detalle y calendario contra datos vivos. Vertical slice de la semana 2: elegir fechas → <code>POST /reservations</code> → ver el cambio de estado por polling.
URLs prefirmadas (Wa)	Check-in/check-out con <code>expo-camera</code> : 1 foto → subir → <code>PATCH /checkin /checkout</code> .
Seeds (Wa)	Pruebas de Mis rentas con todos los estados y del buscador con catálogo variado.
Decisión sobre reporte (equipo)	Botón "Reportar problema" en el detalle de la reserva: <code>POST /report</code> con foto y motivo → depósito congelado.

Resumen en una línea por persona

Trucy:

resolver 4 observaciones + 3 respuestas cortas → congelar contrato.

Wa:

rulesets + CODEOWNERS + spec de URLs prefirmadas + seeds.

Silverk:

items de prueba + acuerdo de visibilidad del reporte.

Zero:

aprobar PRs y subir el scaffold.

RentaTodo · Documento del rol Zero (app móvil) · 14-jul-2026. El contrato válido es el spec OpenAPI congelado; todo lo aquí propuesto se discute en los PRs correspondientes. Scaffold de referencia: rama `feature/mobile-scaffold` · Mockup: `apps/mobile/docs/mock_flujo_arrendatario.html`.